

Министерство образования Республики Беларусь

Учреждение образования  
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ  
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления

Кафедра информационных технологий автоматизированных систем

ОТЧЕТ  
по лабораторной работе №2  
по дисциплине  
«Администрирование и программирование распределенных приложений»

Выполнила:

ст. гр. 320604  
К. А. Хаджинова

Проверила:

Ю. О. Герман

Минск 2026

## СОДЕРЖАНИЕ

|                                                                        |    |
|------------------------------------------------------------------------|----|
| 1 ЦЕЛЬ РАБОТЫ .....                                                    | 3  |
| 2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ .....                                         | 4  |
| 3 ВЫПОЛНЕНИЕ РАБОТЫ .....                                              | 5  |
| 3.1 Подготовка среды и запуск .....                                    | 5  |
| 3.2 Индивидуальное задание .....                                       | 5  |
| 3.3 Результаты работы .....                                            | 9  |
| 3.4 Запуск на нескольких компьютерах .....                             | 13 |
| 4 КОНТРОЛЬНЫЕ ВОПРОСЫ .....                                            | 17 |
| 4.1 Назначение протоколов <i>TCP</i> и <i>UDP</i> и их отличие .....   | 17 |
| 4.2 Объекты и методы отправки данных по сети .....                     | 17 |
| 4.3 Пересылка данных в обратном направлении (от клиента к серверу).... | 17 |
| 4.4 Как определить сетевой адрес своего компьютера .....               | 17 |
| ЗАКЛЮЧЕНИЕ .....                                                       | 18 |

## **1 ЦЕЛЬ РАБОТЫ**

Познакомиться с техникой использования сокетных классов *Java*.

## 2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Сокет (*socket*) – программная точка соединения двух приложений. Для *TCP*-соединения нужны *IP*-адрес компьютера, номер порта, серверный и клиентский сокет. Протокол *TCP* устанавливает соединение, гарантирует доставку данных, сохраняет их порядок и повторно передает потерянные пакеты. Протокол *UDP* работает без постоянного соединения и не гарантирует доставку.

В приложении «клиент-сервер» параллельно работают два потока: поток-сервер и поток-клиент. Сервер создает *ServerSocket* на заданном порту, метод *accept()* блокирует поток до подключения клиента и возвращает объект *Socket*. Клиент создает *Socket*("127.0.0.1", порт), где 127.0.0.1 – адрес локального компьютера. Сервер отправляет текст через *PrintStream* (методы *println* и *flush*), клиент читает строки из входного потока *DataInputStream* методом *readLine()*. Каждый поток реализуется классом, наследующим *Thread*, и запускается методом *start()*. Графический интерфейс приложения строится на классах *AWT*.

## 3 ВЫПОЛНЕНИЕ РАБОТЫ

### 3.1 Подготовка среды и запуск

Приложение написано на *Java* и скомпилировано *JDK 17 (Zulu 17.0.15)* командой `javac -encoding UTF-8 -d out src\serv.java`. Запуск: `java -Dfile.encoding=UTF-8 -cp out serv`. Методические указания используют устаревшие методы *AWT* и *DataStream.readLine()*, при компиляции они выдают предупреждения об устаревших методах, но работают.

### 3.2 Индивидуальное задание

Вариант 2: создать два клиента через потоки. Один клиент добавляет и снимает деньги со счета, второй только снимает. Общее задание: приложение для двух клиентов, работающих с одним счетом, каждому клиенту соответствует свой поток; приложение клиент-сервер с визуальным интерфейсом запускает клиента по нажатию кнопки и отражает состояние счета в текстовом поле.

Реализация: серверный поток *Account* хранит общий счет `static int amount = 200`, слушает порт 3001 и записывает в журнал окна каждый запрос (откуда пришел и каким стал счет). Клиентский поток *clientThread* подключается к серверу по заданному адресу (по умолчанию 127.0.0.1), отправляет запрос *ADD* или *WITHDRAW*, читает ответ вида *Account:<сумма>* и передает его окну. Размер операции генерируется на сервере случайно: `(int)(Math.random() * 1000)`. Окно *serv* содержит три кнопки: «Клиент 1: добавить», «Клиент 1: снять» (клиент 1 добавляет и снимает) и «Клиент 2: снять» (клиент 2 только снимает), текстовое поле счета и журнал операций. Нажатие кнопки запускает новый поток клиента. Адрес сервера задается аргументом запуска, поэтому клиентские окна можно открыть на других компьютерах (раздел «Запуск на нескольких компьютерах»).

#### Листинг 3.1 – Серверный поток *Account*

```
// Серверный поток: хранит счёт и обрабатывает запросы клиентов
class Account extends Thread {
    ServerSocket server;
    String amountstring;
    static int amount = 200;
    serv window; // окно, в журнал которого сервер записывает запросы

    public Account(serv window) {
        this.window = window;
    }

    public void run() {
```

```

try {
    server = new ServerSocket(3001); // Номер сокета
} catch (Exception e) {
    System.out.println("Ошибка соединения" + e);
}
while (true) {
    Socket s = null;
    try {
        s = server.accept(); // ожидание соединения с клиентом
    } catch (Exception e) {
        System.out.println("Ошибка" + e);
    }
    try {
        // Чтение запроса клиента (ADD или WITHDRAW)
        BufferedReader in = new BufferedReader(new
InputStreamReader(s.getInputStream()));
        String request = in.readLine();

        PrintStream ps = new PrintStream(s.getOutputStream()); // PrintStream
предназначен для текстового вывода
        int amountcur = ((int) (Math.random() * 1000));
        if (request.equals("ADD"))
            amount += amountcur;
        else
            amount -= amountcur;
        amountstring = Integer.toString(amount);
        ps.println("Account:" + amountstring); // передача строки клиенту
        ps.flush();
        // Запись в журнал сервера: откуда пришёл запрос и каким стал счёт
        window.log.append("[сервер] " + request + " от " +
s.getInetAddress().getHostAddress() + ", счёт: " + amountstring + "\n");
        window.accountField.setText(amountstring);
        s.close(); // Сокетное соединение закрывается
    } catch (Exception e) {
        System.out.println("Ошибка " + e);
    }
}
}
}
}

```

Сервер последовательно принимает подключения, читает строку запроса через *BufferedReader* (пересылка данных от клиента к серверу), изменяет счет и отправляет ответ клиенту. Код клиентского потока приведен в листинге.

### Листинг 3.2 – Клиентский поток *clientThread*

```

// Клиентский поток: отправляет запрос серверу и читает ответ
class clientThread extends Thread {
    DataInputStream dis = null;
    Socket s = null;
    String name;

```

```

String request;
String host;
serv window;

public clientThread(String name, String request, serv window, String host) {
    this.name = name;
    this.request = request;
    this.window = window;
    this.host = host;
    try {
        s = new Socket(host, 3001);
        dis = new DataInputStream(s.getInputStream());
    } catch (Exception e) {
        System.out.println("Ошибка: " + e);
    }
}

public void run() {
    if (s == null) { // соединение не установлено
        window.log.append(name + ": нет связи с сервером " + host + "\n");
        return;
    }
    try {
        // Отправка запроса серверу
        PrintStream ps = new PrintStream(s.getOutputStream());
        ps.println(request);
        ps.flush();

        String msg = dis.readLine(); // Клиент читает строку из сокета
        if (msg != null) {
            System.out.println(name + ": " + msg);
            window.showMessageDialog(name + ": " + msg);
        }
        s.close();
    } catch (Exception e) {
        System.out.println("ERRORR+" + e);
    }
}
}

```

Окно приложения и запуск клиентов по кнопкам реализованы классом *serv*, код которого приведен в листинге.

### Листинг 3.3 – Окно приложения *serv*

```

// Окно приложения
public class serv extends Frame implements ActionListener {
    Button client1Add = new Button("Клиент 1: добавить");
    Button client1Withdraw = new Button("Клиент 1: снять");
    Button client2Withdraw = new Button("Клиент 2: снять");
    TextField accountField = new TextField("200", 10);
    TextArea log = new TextArea(8, 40);
    String host; // адрес компьютера, на котором работает сервер

```

```

public serv() {
    this("127.0.0.1");
}

public serv(String host) {
    super("Счёты");
    this.host = host;
    setLayout(new FlowLayout());
    add(client1Add);
    add(client1Withdraw);
    add(client2Withdraw);
    add(new Label("Счёт:"));
    add(accountField);
    add(log);

    client1Add.addActionListener(this);
    client1Withdraw.addActionListener(this);
    client2Withdraw.addActionListener(this);

    addWindowListener(new WindowAdapter() {
        public void windowClosing(WindowEvent we) {
            System.exit(0); // Закрывать приложение
        }
    });
}

// Запуск клиента по нажатию кнопки
public void actionPerformed(ActionEvent ae) {
    if (ae.getSource() == client1Add)
        new clientThread("Клиент 1", "ADD", this, host).start();
    else if (ae.getSource() == client1Withdraw)
        new clientThread("Клиент 1", "WITHDRAW", this, host).start();
    else if (ae.getSource() == client2Withdraw)
        new clientThread("Клиент 2", "WITHDRAW", this, host).start();
}

// Отображение состояния счёта
public void showMessage(String message) {
    accountField.setText(message.substring(message.indexOf(':', message.indexOf(':') + 1) +
1));
    log.append(message + "\n");
}

// IP-адреса этого компьютера в локальной сети (для ввода на других компьютерах)
static String localAddresses() {
    String result = "";
    try {
        Enumeration<NetworkInterface> interfaces = NetworkInterface.getNetworkInterfaces();
        while (interfaces.hasMoreElements()) {
            NetworkInterface ni = interfaces.nextElement();
            if (!ni.isUp() || ni.isLoopback())

```

```

        continue;
        Enumeration<InetAddress> addresses = ni.getInetAddresses();
        while (addresses.hasMoreElements()) {
            InetAddress address = addresses.nextElement();
            if (address instanceof Inet4Address)
                result += address.getHostAddress() + " ";
        }
    }
} catch (Exception e) {
    System.out.println("Ошибка " + e);
}
return result.trim();
}

public static void main(String args[]) {
    if (args.length == 0) { // сервер и клиенты на одном компьютере
        serv f = new serv();
        f.setTitle("Счёт (сервер, IP: " + localAddresses() + ")");
        f.setSize(400, 400);
        f.setVisible(true);
        new Account(f).start();
    } else { // только клиенты, сервер на другом компьютере
        serv f = new serv(args[0]);
        f.setTitle("Счёт (клиент, сервер: " + args[0] + ")");
        f.setSize(400, 400);
        f.setVisible(true);
    }
}
}
}

```

### 3.3 Результаты работы

Окно после запуска показано на рисунке 3.1.

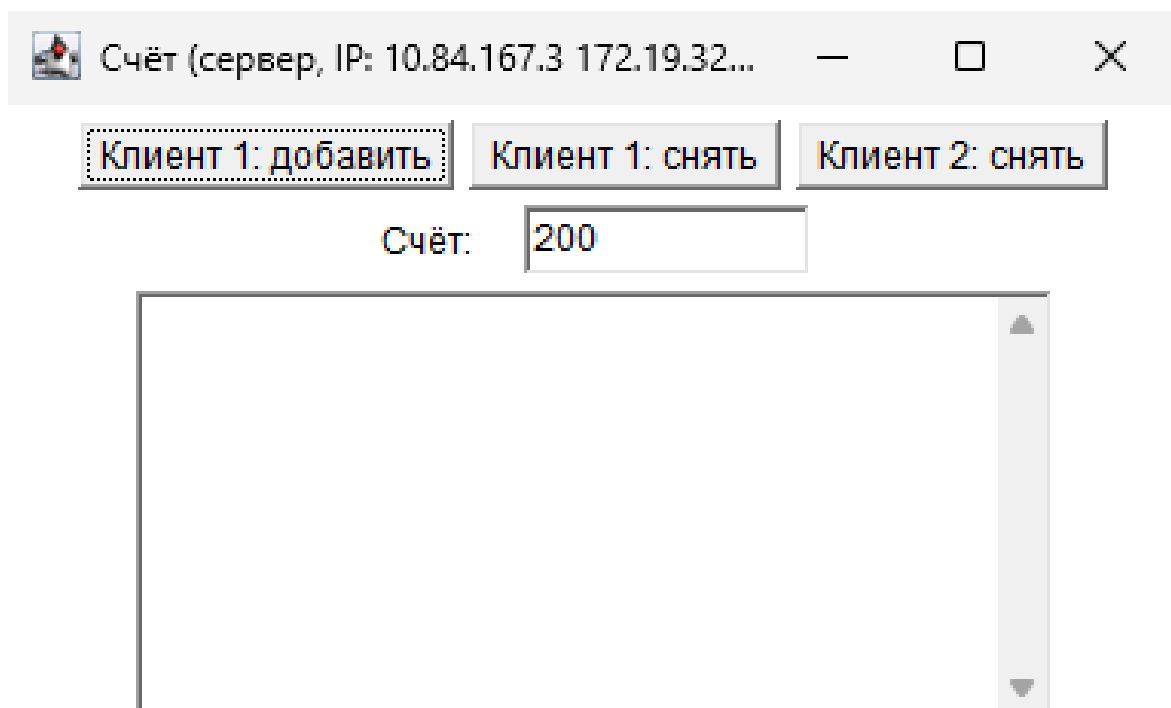


Рисунок 3.1 – Окно приложения после запуска

В поле «Счет» показано начальное значение 200, журнал операций пуст, в заголовке окна показаны *IP*-адреса компьютера-сервера.

Нажатие кнопки «Клиент 1: добавить» запускает поток первого клиента, результат показан на рисунке 3.2.

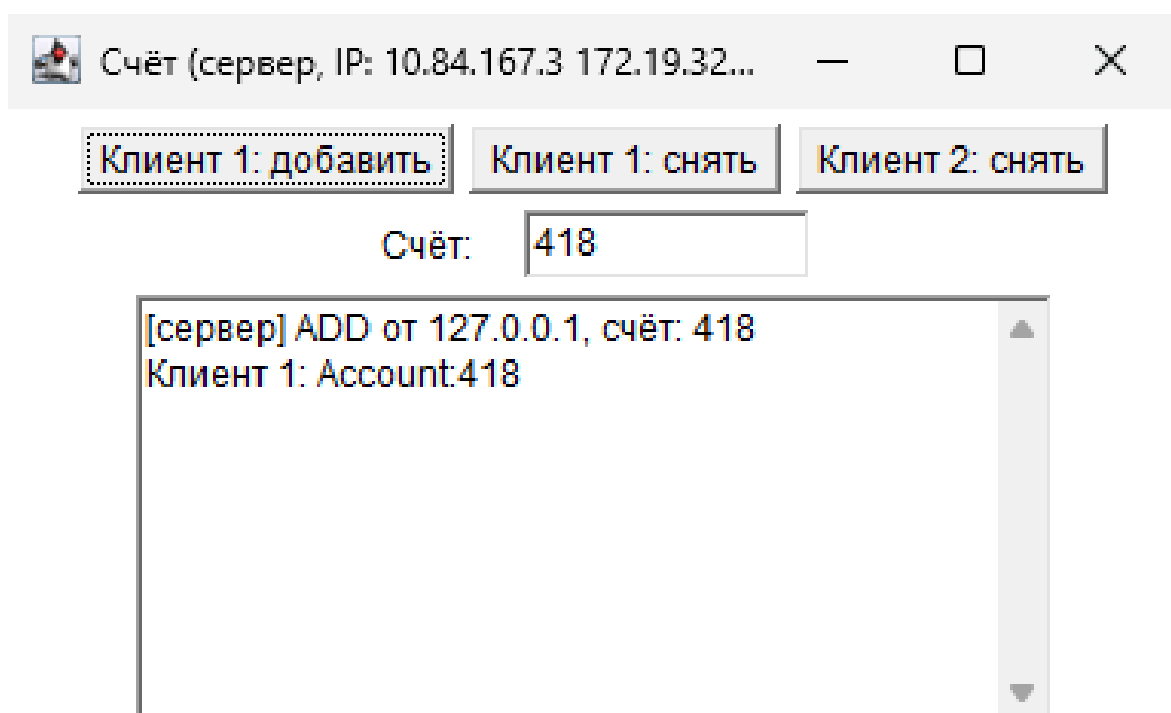


Рисунок 3.2 – Клиент 1 добавил деньги на счет

Сервер прибавил случайную сумму, счет стал равным 418. В журнал добавлены запись сервера и ответ клиента.

Операция снятия клиентом 1 показана на рисунке 3.3.

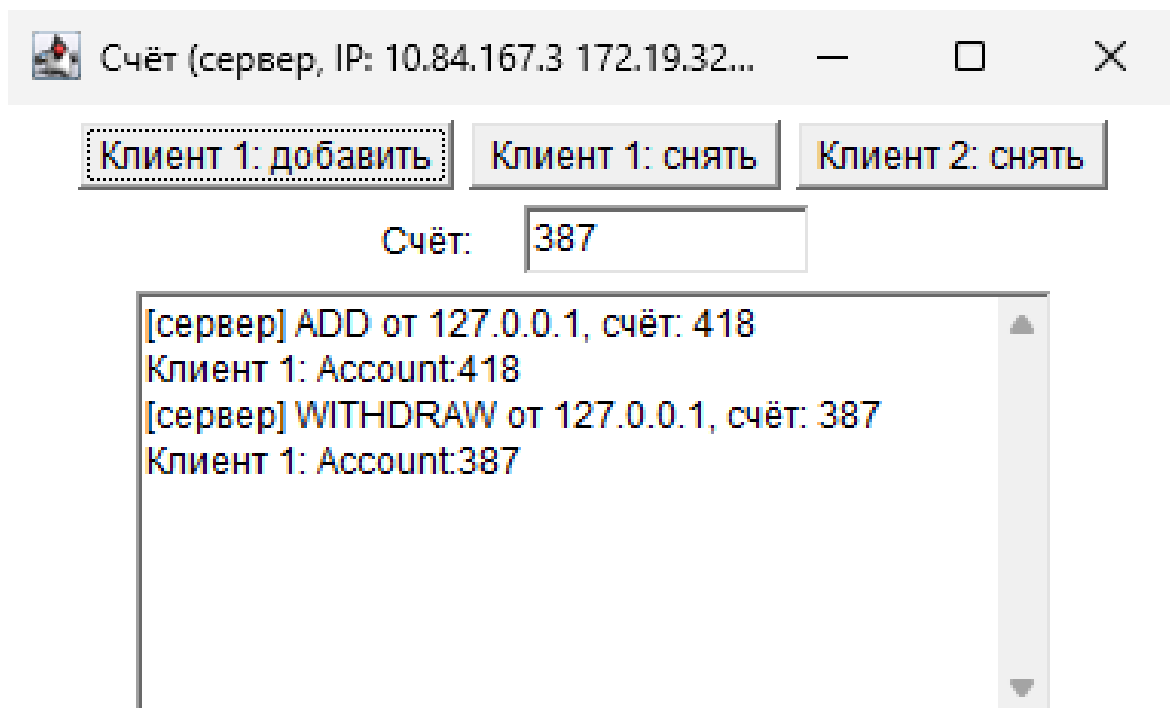


Рисунок 3.3 – Клиент 1 снял деньги со счета

После снятия счет уменьшился с 418 до 387.

Клиент 2 выполняет только снятие, результат показан на рисунке 3.4.

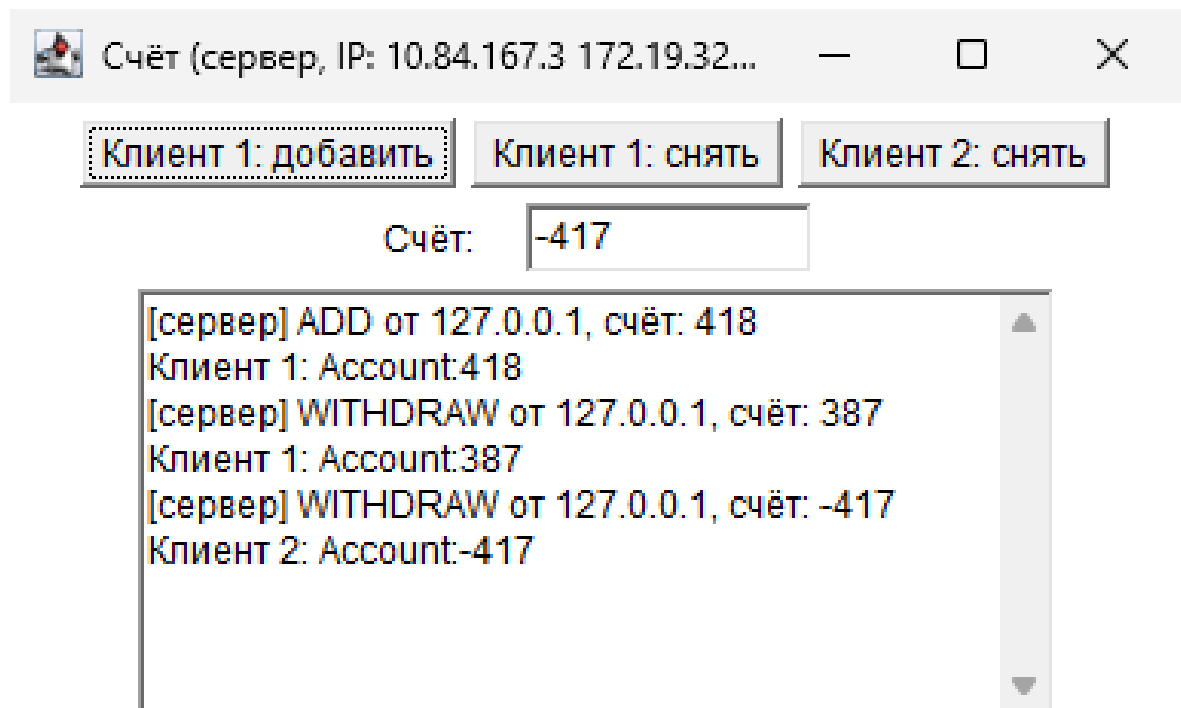


Рисунок 3.4 – Клиент 2 снял деньги со счета

Счет стал равным минус 417: размер операции генерируется случайно, а проверка остатка в задании не предусмотрена. Оба клиента работают с одним общим счетом, в журнале видны операции обоих.

Итоговое состояние после нескольких операций показано на рисунке 3.5.

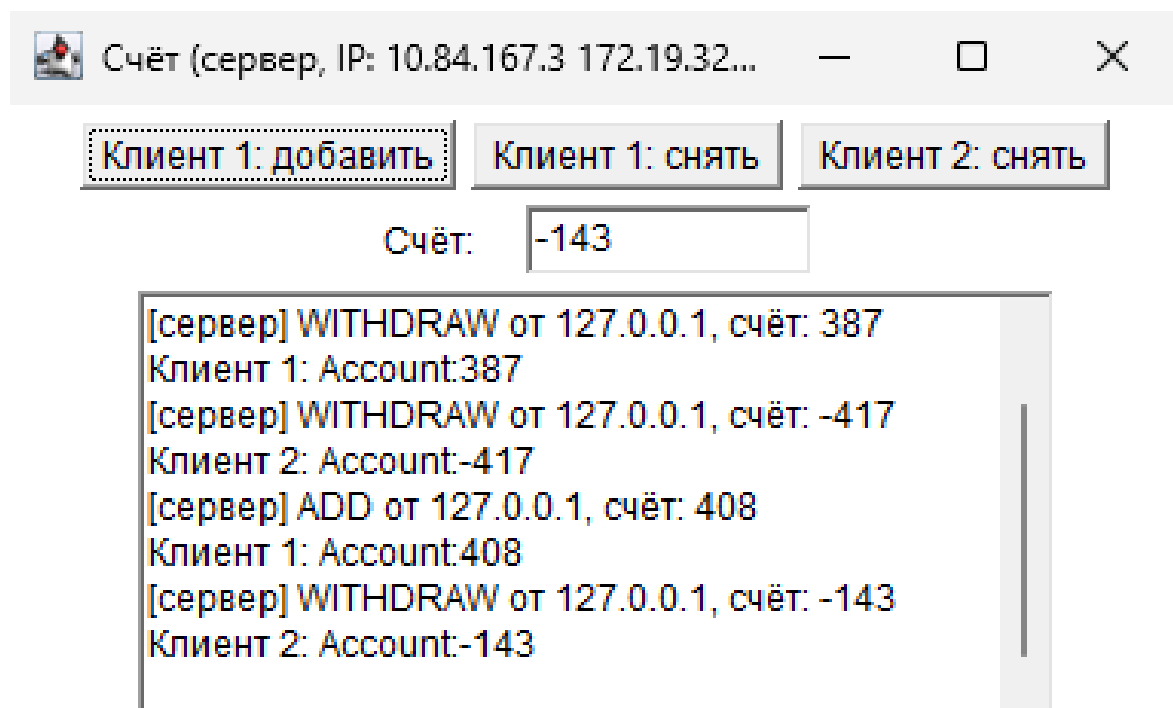


Рисунок 3.5 – Состояние счета после нескольких операций

Последовательность операций клиента 1 (добавление) и клиента 2 (снятие) привела к значению минус 143. Журнал отражает все выполненные операции по порядку: запись сервера (с адресом 127.0.0.1) и ответ соответствующего клиента.

### 3.4 Запуск на нескольких компьютерах

Для защиты приложение запускается на трех компьютерах: сервер на ноутбуке с *Windows*, клиенты на *Mac* и на *Linux* (виртуальная машина *WSL* на этом же ноутбуке). Приложение собрано в файл *serv.jar* (*Java* 11 и новее), способы запуска приведены в таблице 3.1.

Таблица 3.1 – Запуск приложения на разных компьютерах

| Компьютер                            | Команда                                  | Назначение                                                     |
|--------------------------------------|------------------------------------------|----------------------------------------------------------------|
| <i>Windows</i> (сервер)              | <i>Java -jar serv.jar</i>                | Сервер и клиентские кнопки, <i>IP</i> -адреса в заголовке окна |
| <i>Mac</i> или <i>Linux</i> (клиент) | <i>Java -jar serv.jar</i><br>192.168.0.5 | Только клиентские кнопки, адрес сервера в аргументе            |
| <i>Windows</i> (клиент)              | <i>Java -jar serv.jar</i><br>192.168.0.5 | То же на втором компьютере с <i>Windows</i>                    |

Клиент подключается к порту 3001 сервера, поэтому на компьютере с сервером этот порт должен быть открыт в брандмауэре (скрипт *ADMIN-open-*

*port-3001.bat*), а компьютеры должны находиться в одной сети. Проверка выполнена с клиентом в *Linux* (*WSL Ubuntu*) и сервером в *Windows*, в аргументе указан адрес *Windows* с точки зрения *WSL* (172.19.64.1). Окно клиента показано на рисунке 3.6.

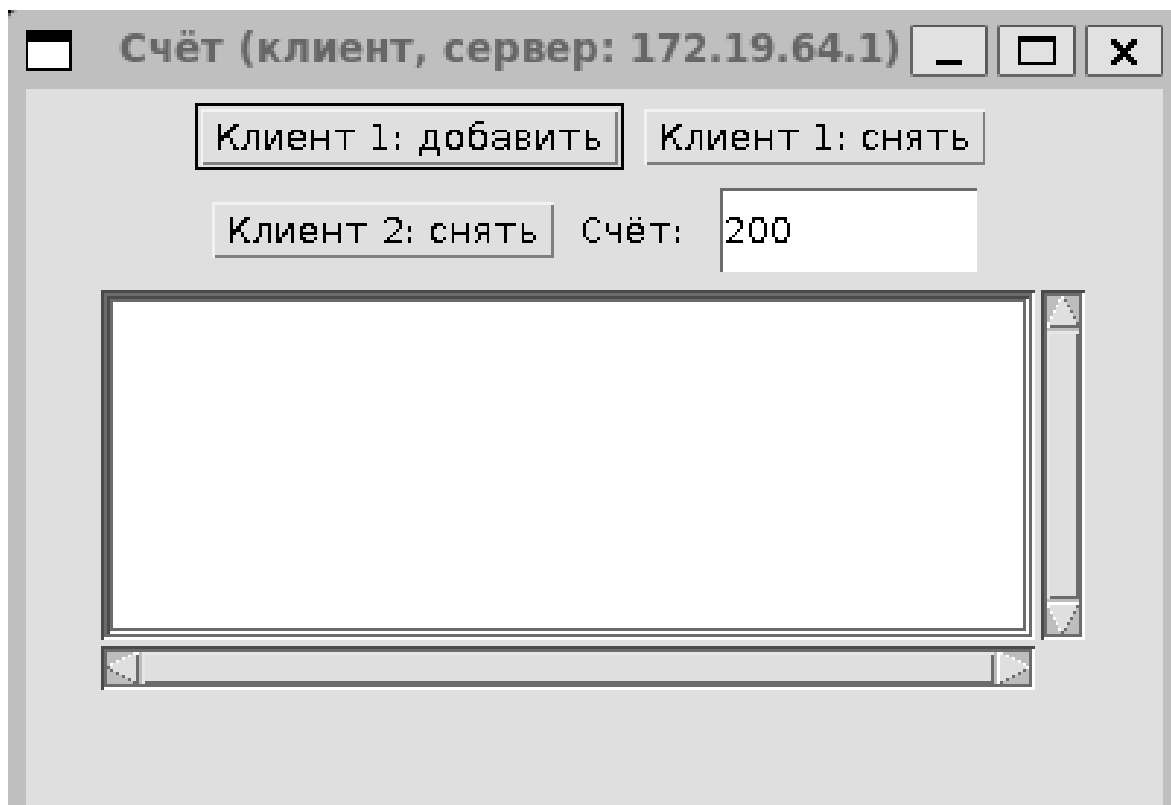


Рисунок 3.6 – Клиент в *Linux* (*WSL*) перед началом работы

В заголовке окна указан адрес сервера 172.19.64.1, начальное значение счета 200, журнал пуст.

После нажатия кнопок клиента счет изменяется на сервере, результат на стороне клиента показан на рисунке 3.7.

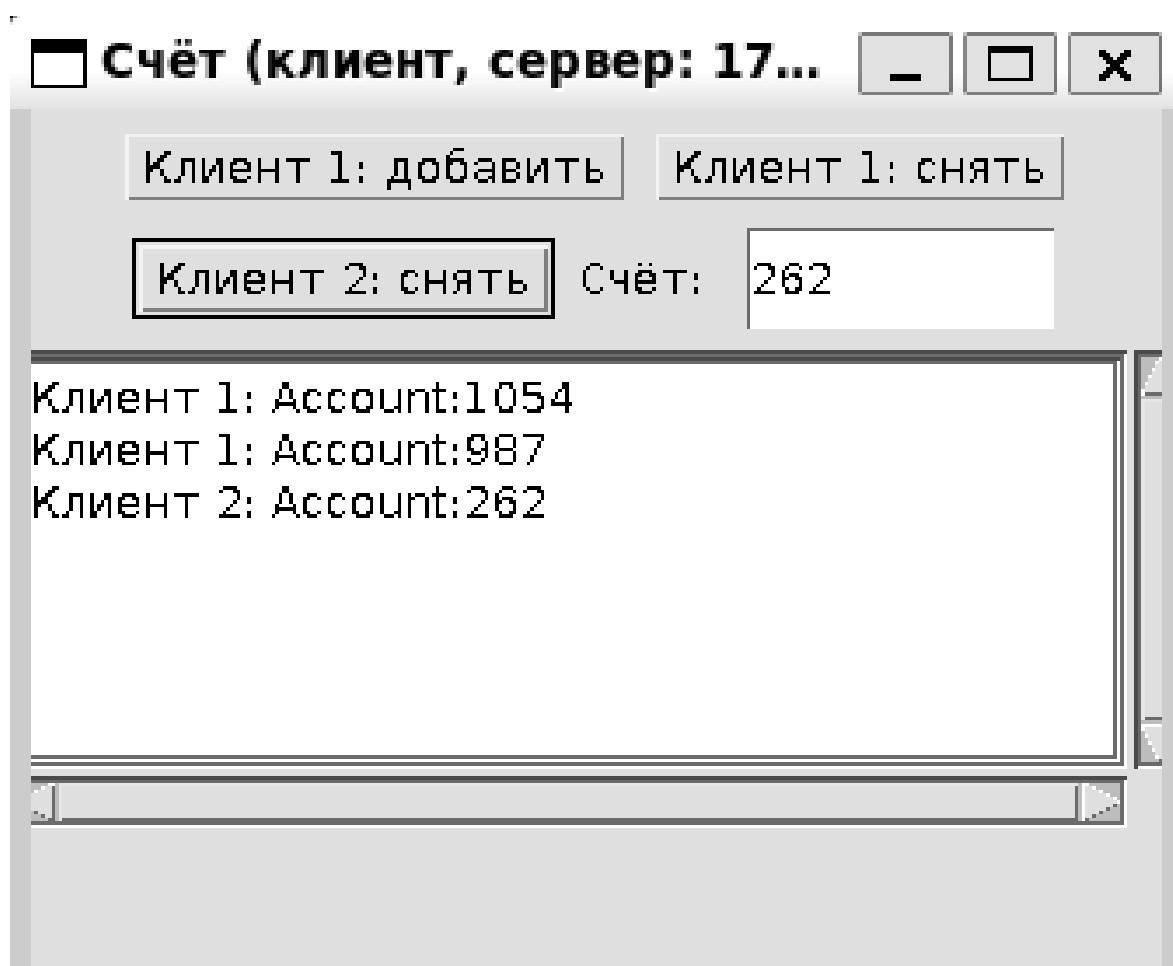


Рисунок 3.7 – Клиент в *Linux* после трех операций

Клиент 1 добавил деньги (счет 1054) и снял (987), клиент 2 снял деньги (262). Все операции выполнены через сеть на сервере в *Windows*.  
Окно сервера после этих операций показано на рисунке 3.8.

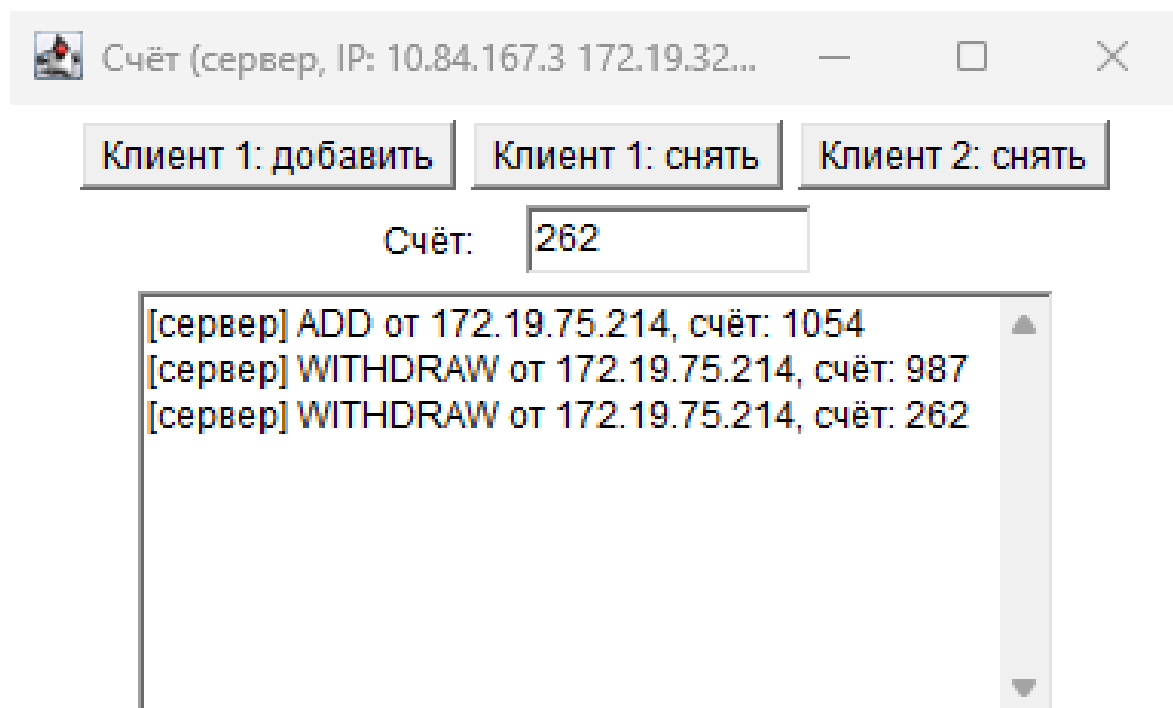


Рисунок 3.8 – Журнал сервера в *Windows*

В журнале сервера записаны три запроса с адреса клиента 172.19.75.214 и значения счета 1054, 987 и 262, совпадающие с ответами клиента.

## 4 КОНТРОЛЬНЫЕ ВОПРОСЫ

### 4.1 Назначение протоколов *TCP* и *UDP* и их отличие

*TCP* – протокол с установлением соединения: он гарантирует доставку данных, сохраняет порядок пакетов и повторно передает потерянные. *UDP* работает без постоянного соединения, не гарантирует доставку и порядок, но имеет меньшие накладные расходы. В работе используется *TCP*, так как применяются классы *ServerSocket* и *Socket*.

### 4.2 Объекты и методы отправки данных по сети

Данные отправляются через выходной поток сокета: *s.getOutputStream()* оборачивается в *PrintStream*, методом *println()* записывается строка, методом *flush()* буфер принудительно отправляется. Читаются данные из *s.getInputStream()* через *DataInputStream* или *BufferedReader* методом *readLine()*.

### 4.3 Пересылка данных в обратном направлении (от клиента к серверу)

Клиент использует *PrintStream* поверх *s.getOutputStream()* и отправляет строку запроса, сервер читает ее через *BufferedReader* поверх *s.getInputStream()* методом *readLine()*. В работе так передаются запросы *ADD* и *WITHDRAW*.

### 4.4 Как определить сетевой адрес своего компьютера

В командной строке *Windows* выполняется команда *ipconfig*, в *Java* адрес возвращает *InetAddress.getLocalHost().getHostAddress()*. Адрес 127.0.0.1 – *loopback*-адрес, обозначающий этот же компьютер.

## ЗАКЛЮЧЕНИЕ

Изучены сокетные классы *Java*: создан серверный поток на *ServerSocket* и клиентские потоки на *Socket*, реализована передача запросов от клиентов к серверу и ответов в обратном направлении. По варианту 2 два клиента работают с общим счетом: клиент 1 добавляет и снимает деньги, клиент 2 только снимает. Приложение с графическим интерфейсом запускает клиента по нажатию кнопки и отражает состояние счета в текстовом поле. Цель работы достигнута.